

2026-09-16

AWS AGENTCORE MASTERCLASS

Foundation & Architecture

- Kaviraj A

Why AgentCore?

AgentCore is AWS's platform to build, connect, and operate AI agents. It provides a modern, secure, serverless runtime with full lifecycle management — designed for production at scale.

Why AgentCore
Bedrock Agents Classic retirement, explained

Architecture Overview
How the services fit together end-to-end

Runtime
Isolation, sessions, deployment model

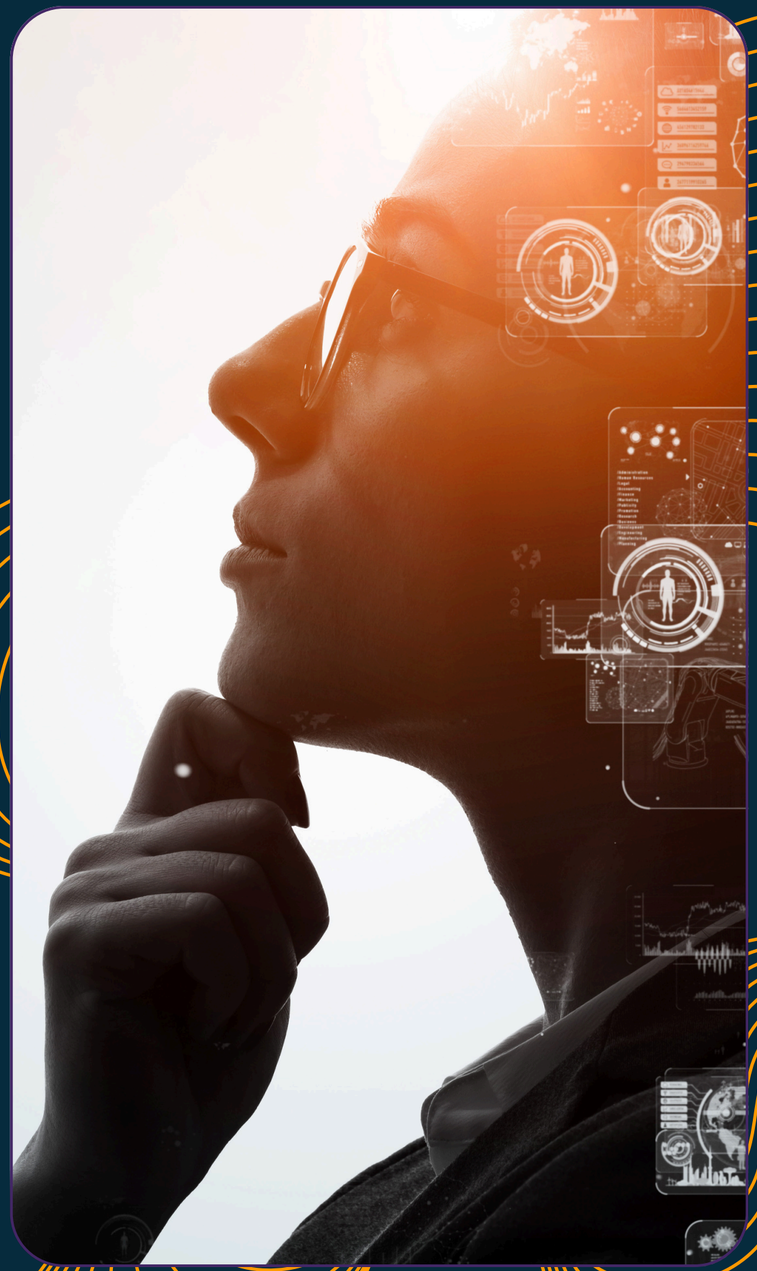
Memory
Short-term vs long-term, strategies

Gateway
MCP tool conversion, auth flow

Identity
Token vault, OAuth delegation

Observability & More
Browser, Code Interpreter, Policy, Registry

Best Practices
Edge cases & live demo



The AgentCore Platform

A layered view — governance sits on top, everything runs on Runtime + Memory

FOUNDATION
The Base Layer

- **Runtime** — Serverless, session-isolated execution
- **Memory** — Short-term + long-term context

CONNECTIVITY
Access

- **Gateway** — Turns APIs & Lambda into MCP tools
- **Identity** — OAuth token vault, scoped delegation

GOVERNANCE
Oversight

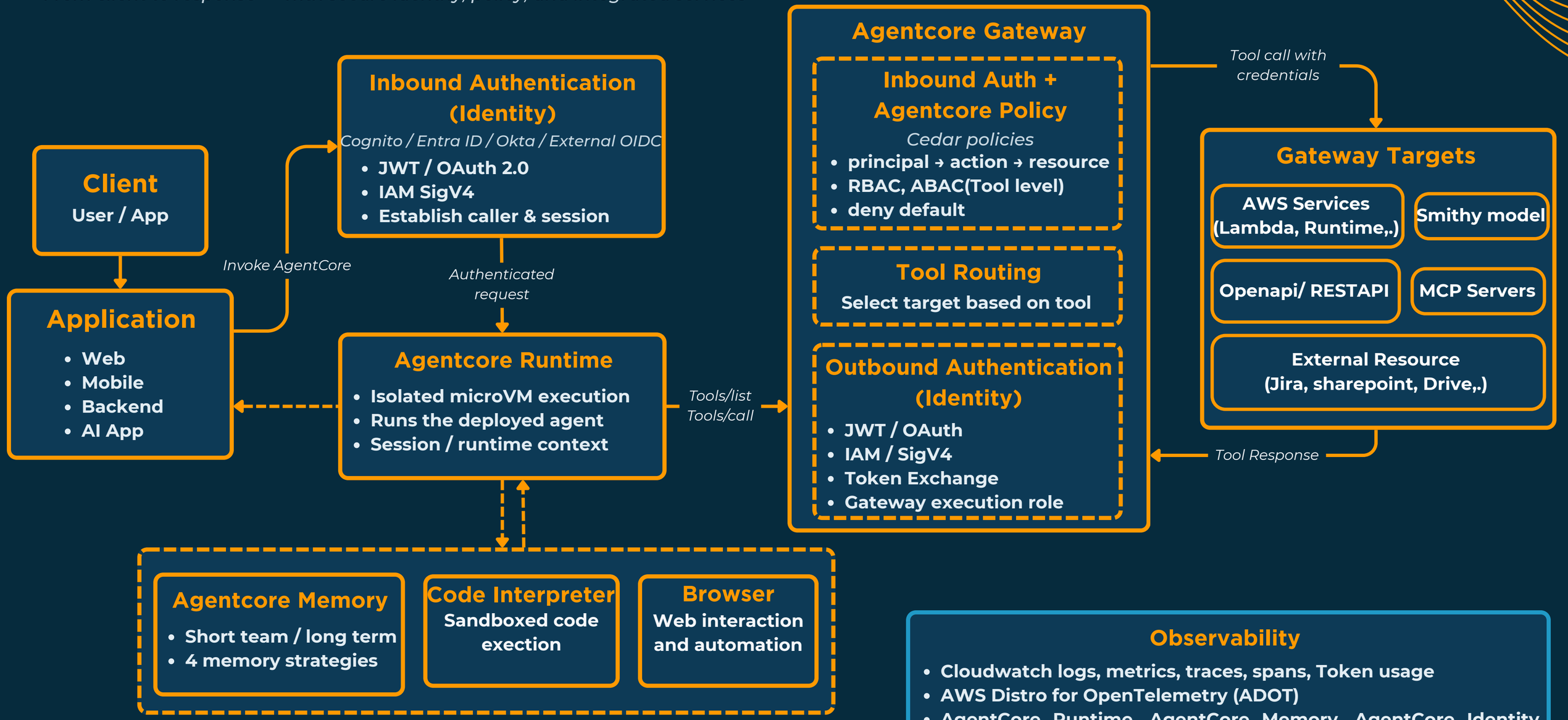
- **Observability** — OTel + CloudWatch
- **Policy** — Cedar guardrails
- **Evaluations** — Quality scoring
- **Registry** — Discovery catalog

TOOLS
Capability

- **Browser** — Managed browser automation sandbox
- **Code Interpreter** — Isolated sandbox for agent-written code

AWS AgentCore — Request Flow & Services

From client to response — with secure identity, policy, and integrated services



Observability

- Cloudwatch logs, metrics, traces, spans, Token usage
- AWS Distro for OpenTelemetry (ADOT)
- AgentCore Runtime, AgentCore Memory, AgentCore Identity and AgentCore Gateway

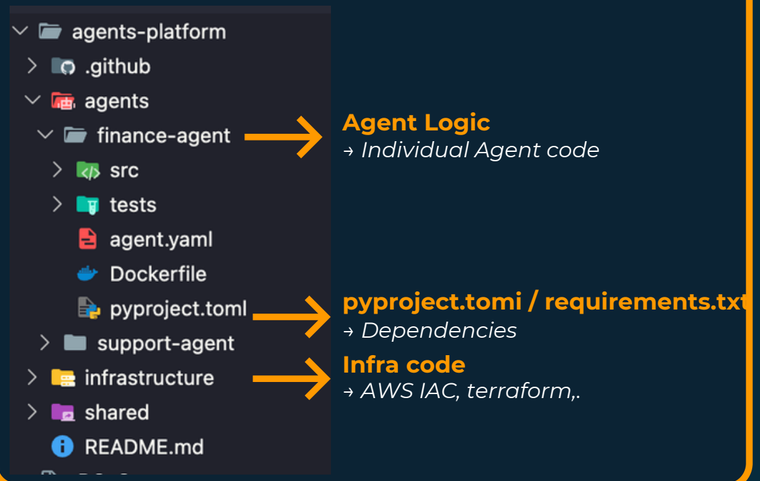
Source: AWS docs — AgentCore Identity, Runtime, Gateway, Policy, Memory, Browser, and Code Interpreter

AgentCore Runtime: Code → Deploy → Invoke

Production project structure, runtime contract, deployment options, and invocation

1 PRODUCTION PROJECT

Organize code, configuration, dependencies & tests separately



2 RUNTIME ENTRPOINT

Sample python code for runtime initialization

```python
from bedrock_agentcore.runtime import BedrockAgentCoreApp

app = BedrockAgentCoreApp()

@app.entrypoint
def agent_invocation(payload, context):
    prompt = payload.get("prompt", "")
    session_id = context.session_id

    result = agent(prompt)

    return {
        "result": result,
        "session_id": session_id
    }
▷ Generate Mermaid Diagram | 💬 Open Chat @mermaid-chart
app.run()
```

EntryPoint
Receives invocation

Payload
Timer

Context
Runtime content

Agent Execution
Invoke application

3 RUNTIME CONTRACT

Caller → *Runtime* → *Context* → *Agent* → *Response*

PAYLOAD
- JSON or binary request body
- Prompt, user input, task parameters
- Validate input before execution
- Maximum payload size: 100 MB

CONTEXT
- Session-aware execution
- context.session_id
- Runtime/request context
- Context supplied by the runtime/SDK

RESPONSE
- Standard response
- Streaming/event-stream response
- Process response chunks
- Client-side error handling

4 DEPLOYMENT LIFECYCLE

1 CREATE
agentcore create
Scaffold project

2 CONFIGURE
agentcore.json
Runtime configuration

3 DEPLOY
agentcore deploy
Deploy live runtime

DEPLOYMENT OPTIONS

CODEZIP
- *Simple / smaller dependencies*
- *Fast development iteration*
- *Managed runtime packaging*

CONTAINER
- *Custom dependencies*
- *OS / system libraries*
- *Larger applications*
- *Existing container CI/CD*

DEVELOPMENT LOOP

agentcore dev
Local development / hot reload / testing

SUPPORTED LANGUAGES

Python · *JavaScript* · *TypeScript*

SDK / FRAMEWORK ECOSYSTEM

Strands · *LangGraph* · *OpenAI Agents SDK* · *Bedrock AgentCore SDK* · *Google ADK*

PRODUCTION AUTOMATION

AWS CDK · *AWS SDK / API* · *CI/CD pipelines* · *Terraform*

AgentCore Runtime — Production Operations

Beyond deployment: scaling, resilience, security, tools, and observability at runtime

LIFECYCLE

- *Idle timeout: 900s default*
- *Max lifetime: 8h for microVMs*
- *Capacity providers can extend*

NETWORK & SECURITY

- *PUBLIC or VPC + IAM*
- *Approved subnets/security groups*
- *IMDSv2 required*
- *Execution roles: least-privilege*

GATEWAY + POLICY

- *MCP tools + Cedar*
- *Cedar: LOG_ONLY → ENFORCE*
- *Default-deny; forbid-wins*
- *Route governed tools via Gateway*

BUILT-IN TOOLS

- *Browser + Code Interpreter*
- *Browser: public net + live view*
- *Code Interpreter: Sandbox/Public/VPC*
- *Roles/timeout/isolation per tool*

SCALING & CONCURRENCY

- *5,000 / 2,500 concurrent sessions*
- *Default quota per account/region*
- *200 interactions/sec; 25 new/sec*
- *Code Interpreter: 25-session cap*

RESILIENCE

- *409 = session already in progress*
- *Retry with backoff on Throttling*
- *StopRuntimeSession: idempotency token*
- *Design for eventual consistency*

OBSERVABILITY

- *CloudWatch + OTel*
- *p95/p99 latency, duration*
- *Token usage, errors, logs, traces*
- *Alarm on SLO drift*

COLD STARTS

- *Provision → init → load deps*
- *Latency varies by deployment type*
- *Reduce size; defer heavy init*
- *Use warm capacity where supported*

SESSION ISOLATION — each runtimeSessionId gets a dedicated microVM; compute is isolated per session

Source: AWS docs — Runtime lifecycle, security, Gateway/Policy, Browser/Code Interpreter, Observability, service quotas, and API error handling

AgentCore Memory

Two tiers: what's happening now vs what's worth remembering



SHORT-TERM MEMORY

Raw, unprocessed events

- *Turn-by-turn conversation buffer within a session*
- *Available synchronously — ListEvents, GetEvent, ListSessions*
- *Restores context if your app or service restarts*
- *TTL configurable via eventExpiryDuration (e.g. 30 days)*

LONG-TERM MEMORY

Extracted, consolidated records

- *Persists across sessions, scoped by namespace*
- *Populated asynchronously by memory strategies*
- *PII stripped from records by default*
- *No strategies configured = no long-term extraction at all*

Source: AWS docs — Amazon Bedrock AgentCore Memory: Building context-aware agents

Four Memory Strategies(Long term)

Each one decides what gets extracted from raw events

SEMANTIC

Extracts standalone facts using vector embeddings for similarity search

e.g. remembers a customer's team size or which software version they run

/facts/{actorId}/

Only processes USER + ASSISTANT messages

```
{
  "semanticMemoryStrategy": {
    "name": "Facts",
    "namespaces": [
      "/facts/{actorId}/"
    ]
  }
}
```

SUMMARY

Maintains a running summary of the conversation, scoped to one session

e.g. captures that a call covered pricing questions and ended in a demo request

/summaries/{actorId}/{sessionId}/

Resets each session - not cross-session by design

```
{
  "summaryMemoryStrategy": {
    "name": "SessionSummary",
    "namespaces": [
      "/summaries/{actorId}/{sessionId}/"
    ]
  }
}
```

USER PREFERENCE

Learns preferences, choices, and communication style over time

e.g. notes a developer prefers Python and a terse, no-fluff tone

/preferences/{actorId}/

Only processes USER + ASSISTANT messages

```
{
  "userPreferenceMemoryStrategy": {
    "name": "Prefs",
    "namespaces": [
      "/preferences/{actorId}/"
    ]
  }
}
```

EPISODIC

Detects complete episodes and reflects on patterns across many of them

e.g. learns that asking for order details first speeds up refund requests

(per-episode, auto-scoped)

Only strategy with a Reflection stage

```
{
  "customMemoryStrategy": {
    "name": "Episodes",
    "namespaces": [
      "/episodes/{actorId}/"
    ],
    "configuration": {
      "episodic": {}
    }
  }
}
```

Source: AWS docs — AgentCore Memory built-in strategies; episodic memory strategy

AgentCore Memory — Production Operations

A minimal example, common pitfalls, and what changes at scale

AgentCore Memory — minimal example

```python
from bedrock_agentcore.memory import MemoryClient

# Initialize AgentCore Memory
client = MemoryClient(region_name="us-east-1")
# Create memory with long-term memory strategies
memory = client.create_memory_and_wait(
    name="SupportAgent",
    strategies=[
        # Extract facts from conversations
        {"semanticMemoryStrategy": {...}},
        # Create conversation summaries
        {"summaryMemoryStrategy": {...}},
        # Capture user preferences
        {"userPreferenceMemoryStrategy": {...}},
        # Capture past experiences/outcomes
        {"episodicMemoryStrategy": {...}}
    ]
)
# Store a conversation event
client.create_event(
    memory_id=memory["id"],
    actor_id="user-123",
    session_id="session-1",
    messages=[
        ("USER", "I prefer concise technical answers."),
        ("ASSISTANT", "Sure, I'll keep them concise.")
    ]
)
# Retrieve relevant long-term memories
memories = client.retrieve_memories(
    memory_id=memory["id"],
    namespace="/",
    query="What does the user prefer?",
    top_k=5
)
```

Common Pitfalls

- ***No Strategy Configured:*** *Memory defaults to strategies=[] you get short-term only until you add one*
- ***Expecting Mid-Session Recall:*** *Long-term memory populates after the session, not during it, don't gate live logic on it*
- ***Missing Actor/Session Scoping:*** *Skip namespace scoping and one user's facts can leak into another's context*
- ***Stuffing All Retrieved Memories into the Prompt:*** *Always limit to top-k results — dumping everything dilutes signal and burns tokens*
- ***Letting Summaries Go Stale:*** *A hallucinated or dropped fact in a summary compounds silently over future sessions*
- *Only USER + ASSISTANT Roles Get Extracted — facts buried in TOOL messages need the assistant to restate them*

THROUGHPUT

- *200 TPS — CreateEvent / Account*
- *5 TPS — CreateEvent / Session (conversational)*
- *10 TPS — CreateEvent / Session (non-conversational)*
- *30 TPS — RetrieveMemoryRecords*
- *150K Tokens/min — Long-Term Extraction*

QUOTAS & LIMITS

- *150 Memories / Region / Account*
- *6 Strategies / Memory*
- *10 MB Max Event Size*
- *100 Messages / Event*
- *7–365 days Event Retention*

Source: AWS docs — AgentCore Memory building context-aware agents; namespace design patterns; Agent Memory design guide

Gateway Fundamentals: Concepts & Request Flow

What it is, the problems it solves, and how every request actually flows through it

GATEWAY

Central routing + auth layer, every agent, tool, and model call passes through here

Lambda	OpenAPI	Smithy	MCP Server	Connectors
Your custom business logic, in any language	*Turn existing REST APIs into tools instantly*	*Type-safe API models, AWS-native*	*Federate other MCP servers behind one gateway*	*1-click: Slack, Jira, Salesforce, Zendesk, Asana*

WHAT GATEWAY SOLVES

- **Tool auth complexity:** SigV4, OAuth, API keys, JWT, unified
- **Tool silos:** no cross-agent tool sharing without a Gateway
- **Credential overhead:** no more per-agent secret rotation or IAM roles
- **Provider lock-in:** swap LLM providers without touching agent code

KEY CAPABILITIES

- **MCP tool publishing:** Lambda, APIs & MCP servers become agent-ready tools
- Centralized credential management via Identity's token vault
- Cedar policy enforcement + cross-agent tool discovery
- **Unified LLM router:** Bedrock, OpenAI & Anthropic through one /inference endpoint

ARCHITECTURE COMPONENTS

MCP Client (agent-side)
Runs inside the agent process — discovers tools and calls them over MCP

Gateway (central layer)
Routes requests, terminates inbound auth, evaluates policy, brokers outbound creds

Credential Providers & Targets
SigV4 / OAuth 2.0 / API Key / Custom JWT:
Lambda, REST APIs, MCP servers, or LLM providers

REQUEST FLOW

1. Agent calls Gateway with SigV4 or a JWT
2. Gateway evaluates the request against attached Cedar policy
3. Gateway calls the target using its own brokered credentials
4. Target sees the Gateway's identity, never the agent's
5. Every hop is logged to CloudTrail for a full audit trail

Every target falls into one of 3 categories: MCP (aggregated tools), HTTP (agent/A2A passthrough), or Inference (LLM routing)

Source: AWS docs — Gateway core concepts; Supported gateway targets; Inference targets; Gateway service quotas

Building a Gateway: From Setup to First Tool Call

From create_gateway to a working MCP tool call — setup steps, config, and code pattern

1. CREATE
name, roleArn, protocolType=MCP, authorizerType

→

2. REGISTER TARGET
Lambda/API/MCP + tool schema + credential provider

→

3. DISCOVER & CALL
Agent's MCP client lists & invokes tools

CONFIGURATION OPTIONS

- **authorizerType:** AWS_IAM, CUSTOM_JWT, AUTHENTICATE_ONLY, or NONE (dev/test)
- **credentialProviderType:** GATEWAY_IAM_ROLE, OAUTH, API_KEY, CUSTOM_JWT
- **Tool schema:** name, description, inputSchema (JSON Schema)
- protocolType=MCP enables discovery; HTTP targets skip it

CODE PATTERN

```
# Create Gateway
gateway = client.create_gateway(name, roleArn, protocolType="MCP", ...)
# Add MCP target
client.create_gateway_target(gatewayIdentifier, targetConfiguration)
# Connect MCP client
async with streamablehttp_client(gateway["gatewayUrl"], auth=SigV4Auth(...))..
# Discover Gateway tools
tools = await session.list_tools()
# Invoke a tool
result = await session.call_tool("tool_name", {"key": "value"})
```

EDGE CASES

- Tool naming: **every tool is {targetName}___{toolName}**
- MCP session lives in process memory, hold it globally, don't rebuild per request
- authorizerType must match how callers actually authenticate
- credentialProviderType must match the target's real auth need, or calls fail at runtime

TARGET CONFIG KEYS

- Lambda target → targetConfiguration.mcp.lambda + function ARN
- REST API → targetConfiguration.mcp.openApiSchema (inline or S3 spec)
- Existing MCP server → targetConfiguration.mcp.mcpServer (federated)
- API Gateway REST API → targetConfiguration.mcp.apiGateway

Optional: Request/Response Interceptors — Lambda hooks that transform requests, inject credentials, or enforce fine-grained access before/after each tool call

Source: AWS docs — CreateGateway / CreateGatewayTarget API reference; Gateway quick start; target configuration guides

Gateway in Production: Policy, Security & Pitfalls

Cedar policy, user-delegated access, and the checklist before you ship

User-Delegated Access (On-Behalf-Of)

```python
client = GatewayClient(region_name="us-east-1")

# Create Gateway with OAuth/Cognito authentication
auth = client.create_oauth_authorizer_with_cognito("SupportGateway")

gateway = client.create_mcp_gateway(
    name="SupportGateway",
    authorizer_config=auth["authorizer_config"],
    enable_semantic_search=True
)

# Connect a Lambda function as an MCP tool
client.create_mcp_gateway_target(
    gateway=gateway,
    name="ProfileTools",
    target_type="lambda",
    target_payload={
        "lambdaArn": "arn:aws:lambda:us-east-1:123456789012:function:UpdateProfile",
        "toolSchema": {
            "inlinePayload": [
                # MCP tool definitions
            ]
        }
    }
)

print("Gateway ready:", gateway["gatewayUrl"])
```

Production Checklist

- Define Tool Schemas First

  Lock down input/output schemas before deploy — Cedar validates policies against them

- Scope the Gateway IAM Role

  Grant only the target permissions the Gateway execution role actually needs

- Enable CloudTrail + Test Policy

  Full audit trail on every tool call, plus a Cedar enforcement test before go-live

- Monitor Errors + Latency

  Track Gateway error rate and p95/p99 latency per tool, not just overall

TIP: Inference targets route Bedrock/OpenAI/Anthropic calls through this same Gateway — one endpoint, one policy layer, no vendor lock-in

Source: AWS docs — Understanding Cedar policies; Policy scope; Fine-grained access control for Gateway; On-behalf-of token exchange with AgentCore Identity

AgentCore Identity

The IAM layer built for agents, not humans — the fundamentals, plus what's new

WORKLOAD IDENTITY

Created automatically for every Runtime you deploy, no setup required

The agent's own identity, separate from any human user, like a service account for your code

OAUTH CREDENTIAL PROVIDERS

One-time setup per external service (Google, Slack, GitHub, custom)

Handles token refresh automatically — you never write rotation logic yourself

TOKEN VAULT

Encrypted storage for OAuth tokens and API keys, think of it as a secure vault your agents

Tokens never touch your agent's code — your code just asks for one when it needs it

Managed Consent Portal (New)
You used to build your own webpage for users to approve access. Now AWS hosts that consent page for you automatically.

Private Key JWT Auth (New)
Instead of proving identity with a plain secret (like a password), the agent signs a cryptographic token — much harder to steal.

Secrets Manager Integration (New)
Already storing credentials in AWS Secrets Manager? Point a Credential Provider straight at them instead of duplicating secrets.

Service-Linked Role
AWS now creates and manages the IAM permissions each agent needs automatically, no hand-written policy per agent.

Source: AWS docs — AgentCore Identity: authentication and authorization; AgentCore release notes; Managed consent portal announcement (Sep 2026)

OAuth Consent Flow & Token Exchange

How a user grants your agent access to their data

STEP 1
REQUEST

Agent requests access to a protected resource

STEP 2
AUTH URL

AgentCore Identity generates an authorization URL

STEP 3
CONSENT

User navigates to the URL and approves access

STEP 4
TOKEN FETCH

AgentCore Identity retrieves the access token

STEP 5
VAULT STORE

Token is encrypted and stored for reuse

Limitation: Always verify the consenting user matches who initiated the request — forwarded auth URLs can leak access

On-Behalf-Of Token(OBO) Exchange

USER + AGENT TOKEN
The token your agent gets already proves both who the user is and which agent is asking

→

WORKLOAD TOKEN
AgentCore Identity swaps it for a new token, scoped just for the next call downstream

→

DOWNSTREAM TOKEN
That token only works for one target, the user never sees another approval screen

The exchanged token carries BOTH identities, enabling zero-trust authorization at every hop, no re-consent needed

Source: AWS docs — OAuth 2.0 authorization URL session binding; On-behalf-of token exchange with AgentCore Identity

Auth Flows: 2LO vs 3LO (with Code)

Two flows, two very different trust models

Auto-Injected Access Tokens

```python
from bedrock_agentcore.identity.auth import requires_access_token

@requires_access_token(
    # name of the configured credential provider
    provider_name="google-drive-provider",
    # OAuth2 scopes to request
    scopes=["https://www.googleapis.com/auth/drive.readonly"],
    # inject the fetched token into this param name
    into="access_token"
)
def list_files(access_token: str):
    # access_token injected automatically
    return call_google_drive_api(access_token)
```

2LO — CLIENT CREDENTIALS

Machine-to-machine, no human in the loop

- *("2LO" = 2-Legged OAuth, just two parties: your agent and the API)*
- *Agent authenticates as itself, not a user*
- *No consent screen, approved once at setup*
- *Best for: scheduled jobs, backend automation*

3LO — AUTHORIZATION CODE

User-delegated, requires explicit consent

- *("3LO" = 3-Legged OAuth, adds the human user as a third party)*
- *Agent acts on behalf of a specific signed-in user*
- *User sees a consent screen the first time*
- *Best for: "connect your Google Drive" style features*

Source: AWS docs — Obtain OAuth 2.0 access tokens; AgentCore Identity SDK

AgentCore Observability

Zero-code visibility by default — features, edge cases, and how to run it in production

AWS Distro for OpenTelemetry (ADOT)
Auto-instruments Bedrock calls, tool executions

→

CLOUDWATCH
Ingests metrics, traces, and logs

→

GENAI OBSERVABILITY
Dashboards for agents, sessions, and traces

FEATURES

- *3 telemetry layers — dashboards, distributed traces, structured logs*
- *Standardized OTEL format — same telemetry, any framework or model*
- *Export to Datadog, Grafana, or Elastic — no extra instrumentation needed*
- *Rich metadata tagging via OTEL baggage — filter by user, session, experiment*

EDGE CASES

- *Missing **Transaction Search** = empty dashboards (enable once per Region)*
- *DISABLE_ADOT_OBSERVABILITY=true silently turns off auto-instrumentation*
- *OTEL_PYTHON_DISABLED_INSTRUMENTATIONS excludes noisy packages*
- *Non-Runtime agents (EC2, EKS, Lambda, on-prem) get identical capabilities*

PRODUCTION IMPLEMENTATION

- *Alarm on error rate > 5%, P95 latency > 3s, and token usage per session*
- *Find slow tools: fields @timestamp, ToolName, ToolLatency | sort ToolLatency desc*
- *Watch session count as a trend — a sudden drop usually means upstream auth failure*
- *Instrument agents, memory, and gateway together for full-path visibility*

BEST PRACTICES

- *Start Simple, Then Expand — defaults already capture model calls, tokens, tools*
- *Alert proactively with CloudWatch alarms — don't wait for users to report problems*
- *Tag sessions/users/experiments with OTEL baggage for fast filtering*
- *Pair with AgentCore Evaluators & Insights for continuous quality checks*

Source: AWS docs — AgentCore Observability quickstart; Optimizing production agents with AgentCore Observability; Debugging production agents with AgentCore Observability

Instrumenting Python for Observability

How to add it to an existing Python application — no agent-logic changes required

Instrument in One Line

Works identically on EC2, EKS, Lambda, on-prem, or other clouds

1. Install packages

Add to `requirements.txt` (or `pyproject.toml` if using UV):

```
pip install "aws-opentelemetry-distro>=0.10.0" boto3
```

2. Configure environment variables

Create/update your `.env` file:

```
# Enable AgentCore observability
AGENT_OBSERVABILITY_ENABLED=true

# AWS Distro for OpenTelemetry
OTEL_PYTHON_DISTRO=aws_distro
OTEL_PYTHON_CONFIGURATOR=aws_configurator

# OTLP export
OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf

# CloudWatch Logs destination
OTEL_EXPORTER_OTLP_LOGS_HEADERS=x-aws-log-group=<log-group>,x-aws-log-stream=<log-stream>,x-aws-metric-namespace=<namespace>

# Identify your agent in AgentCore Observability
OTEL_RESOURCE_ATTRIBUTES=service.name=<agent-name>
```

3. Update the Dockerfile run command

```
# Local run equivalent: opentelemetry-instrument python app.py
CMD ["opentelemetry-instrument", "python", "main.py"]
```

4. Tag sessions for filtering (optional)

```
from opentelemetry import baggage, context

ctx = baggage.set_baggage("session.id", "user-session-123")
context.attach(ctx)
```

Three Dashboard Views

AGENTS VIEW
Lists every agent, runtime and non-runtime

SESSIONS VIEW
All sessions for a selected agent

TRACES VIEW
Click a trace to explore its full timeline

Core concepts

- ***Session:*** *The entire conversation or user interaction*
- ***Trace:*** *the full journey of one request within that session*
- ***Span:*** *a single step within it (one LLM call, one tool run)*
- ***Metric:*** *an aggregated number over time*
- ***Log:*** *a timestamped event record*

Source: AWS docs — AgentCore Observability quickstart; Build trustworthy AI agents with AgentCore Observability; Monitor on-premises and multi-cloud AI agents

AgentCore Built-in Tools & Governance

Code Interpreter, Browser Tool, Policy, and Agent Registry — the complete picture

Code Interpreter

- *Runs Python, JavaScript, TypeScript*
- *900s default session, up to 8h max*
- *30-day data retention (TTL) after session ends*
- *Sandbox / Public / VPC network modes*
- *Dedicated microVM — sanitized on session end*

Edge Cases

- *25 concurrent sessions per account/region, separate*
- *Real failures: execution timeouts, memory limits, package install restrictions*
- *A real sandbox bypass was found and patched by AWS (2026)*

Browser Tool

- *Remote, containerized browser, not local*
- *Live View (AWS DCV) for real-time monitoring*
- *Session recording to S3 + full replay (DOM, console, network, CDP events)*
- *Same microVM isolation as Code Interpreter*

Edge Cases

- *CAPTCHA has no bypass — hand off to a human via Live View*
- *CORS errors are common when calling a custom Runtime server*
- *Cookies expire per site rules, not configurable by you*

Policy

- *Cedar-based, attaches to the Gateway only*
- *Modes: LOG_ONLY → ENFORCE*
- *Default-deny; forbid-wins*
- *Lambda interceptors run at REQUEST + RESPONSE stages*

```
permit (
    principal,
    action == Tool::"get_patient_record",
    resource
)
when {
    principal.jwt_claims.scope.contains("fhir:read"
};
```

Edge Cases

- *REQUEST interceptor runs before Cedar, it can alter what Cedar evaluates*

Agent Registry

- *Catalog: MCP servers, tools, agents, skills, custom resources*
- *Publish → Approval workflow → Discovery plane*
- *URL-based auto-discovery pulls schemas from live endpoints*
- *Org-wide auto-detection addresses "Shadow AI"*
- *Supports both IAM and OAuth (Custom JWT) inbound auth*

Edge Cases

- *Preview "bedrock-agentcore" namespace retires Sep 17, 2026 - migrate to "agent-registry" now*
- *Hybrid search + native MCP endpoint for AI-agent discovery*

Source: AWS docs — Code Interpreter session management, Browser Tool troubleshooting, Policy and Lambda interceptors, AWS Agent Registry

Live Demo

The Order-Refund Agent, End to End

A customer asks for a refund — five services, one flow, live in under 60 seconds

- *Two live sessions, side by side — zero state leaks, zero infra pre-provisioned*
- *“I see you ordered a [product] on [date]…” — recalled, never told*
- *Two refund requests, same path — one approved, one denied, live*
- *The refund call carries a scoped, consented token — never the agent's own credentials*
- *Flip to Traces — the exact span tree, proving every claim just made*



Thank You

AWS AgentCore Masterclass 2026

What We Covered

- Learned all 9 AgentCore services end to end
- Watched one agent orchestrate five services, live
- Covered MCP, A2A, Identity, and Policy along the way

> *“The line between an agent and an application is getting blurrier.”*
>
> **— Deepak Singh, AWS VP of Databases and AI**

Slido Quiz & Feedback

Slido Quiz



Share Feedback



Thank you for attending!